

Screen A: implementation

Six prompts and one form turned four design documents into a working screen: backend, database, frontend, 133 tests, two pull requests — and the harness improvements the earlier steps had only proposed.

Presented alongside `ScreenA_Implementaion_edited.mp4` (5 min 59 s, cut from 50 min 44 s). Step numbers on the slides match the captions in the video.

Recorded 2026-09-18 · Claude Code (Opus 5) in the JetBrains Rider terminal, auto mode on.

Domain · Application

Infrastructure · Migrations

Api · OpenTelemetry

frontend/production-orders

tests/e2e (Playwright)

plan.md · test-plan.md · evidence.md

ai/improvements/0001 · 0002

Fifty minutes of recording — and one 34-minute turn

The shape is unlike the other three steps. Three quarters of the agent's working time is a single unattended turn, so the human's work sits at both ends: the plan before it, the merge after it.

5

prompts in the cut

two of them corrections

34 m

longest single turn

domain → E2E, unattended

133

tests written

49 + 38 + 38 + 8

2

pull requests

Screen A and the harness

The human

- Chose the E2E tool, the accessibility approach, the git scope and where harness changes go.
- Made Claude restore the superseded plan — twice, until it was in chronological order.
- Approved revision 2, then squash-merged both PRs personally. **Merge was never authorized for the agent.**

The agent, guided by the harness

- Wrote RFC 0002 with its own evaluation case, and had already applied RFC 0001 before the cut begins.
- Named its own deviations: the index, the Docker workaround, an extension method that departed from the DD.
- Proved the restricted login instead of asserting it, and prevented a CRLF failure before it happened.
- Stopped at the merge gate and reconciled `master` afterwards.

11 steps, one video

Each step is one or more clips in the edited video. **Amber: the human acts.** **Teal: the agent works and produces output.**

STEP 0100:00 Four questions E2E · a11y · git scope · harness PR	STEP 0200:32 Plan revision 2 14 steps, and its own caveats	STEP 0301:16 Keep the old plan Restored — in the wrong place	STEP 0401:42 Chronological order The rule fixed, then the file
STEP 0502:08 Approved + RFC 0002 Keep every plan revision	STEP 0602:32 Branches, worktrees Two branches, PRs to come	STEP 0702:44 Backend + database Tests, CI verdict, CRLF caught	STEP 0803:29 API and frontend A real bug found and fixed
STEP 0903:58 E2E and the proof 8 journeys; PR opened; 34m turn ends	STEP 1005:17 The screen exists Both PRs squash-merged	STEP 1105:35 Reconciling master Two things need your call	NOT IN THE CUT The session also opened by applying RFC 0001 from the previous step — see slide 13

Times are positions in `ScreenA_Implementaion_edited.mp4`. 50 min 44 s of source, jump-cut to 5 min 59 s; nothing is sped up. The cut begins at the implementation planning — the session’s opening minutes, which applied the previous step’s RFC 0001, are left to that step’s deck.

The choices the agent would not make alone

E2E TOOL

DEC-025

Which end-to-end test tool?

Playwright · Cypress · none for now

A11Y CHECK

DEC-026

How should accessibility checks run?

axe in Vitest + E2E · manual only

GIT SCOPE

DEC-027

Which git operations are authorized?

branch, commit, push, open PR — **merge: no**

HARNESS PR

DEC-028

Where does the harness change go?

Its own branch/PR · same as Screen A

← ☒ E2E tool ☐ A11y check ☐ Git scope ☐ Harness PR ✓ Submit →

| Which end-to-end test tool should Screen A use? No E2E tool has been chosen yet; WI-001 deferred it.

1. Playwright (Recommended)
@playwright/test driving Chromium against the Docker Compose stack. Covers the create → edit and discard-dialog journeys; can also run automated accessibility (axe) checks.
2. Cypress
Also works well, but only runs JavaScript in the browser and has weaker multi-tab support.
3. No E2E for now
Rely on unit + integration tests; record E2E as not run. The DD's E2E test scenarios stay uncovered.
4. Type something.

5. Chat about this

Enter to select · Tab/Arrow keys to navigate · Esc to cancel

Video 00:06. "No E2E tool has been chosen yet; WI-001 deferred it." — the question carries its own history.

A plan is a document, not a promise

14 numbered steps, each with what it depends on, which skill it uses, what it must deliver and **how that will be checked**. This is the artifact the rest of the session is measured against.

Deliverables and milestones

#	Milestone / step	Depends on	Skill used	Deliverable	Verification method
1	Harness branch: create worktree <code>../WMS-worktrees/harness-dd-companions</code> on <code>feature/harness-dd-companions</code> from <code>master</code> ; copy the RFC 0001 changes (<code>ai/templates/**</code> , <code>ai/skills/detailed-design/SKILL.md</code> , <code>ai/evaluations/baseline-cases.md</code> , <code>ai/improvements/**</code>); commit	none	harness-improvement	commit on the harness branch	<code>git diff master --stat</code> shows only <code>ai/</code> files
2	Screen A branch: create worktree <code>../WMS-worktrees/WI-002</code> on <code>feature/WI-002-production-order-screen</code> from <code>master</code> ; copy the WI-002 design files (work item, docs, <code>CLAUDE.md</code> , READMEs, ADR-0002/DB-	none	implementation	commit on the WI-002 branch	Diff contains only WI-002 design files

The plan itself

14 steps, each with a deliverable and a check

Video 00:55. Steps 1 and 2: the two branches — created before a line of Screen A is written.

THINGS TO KNOW BEFORE APPROVING

CI — opening the PRs starts GitHub Actions; the project's first real run. **E2E stays out of CI** — it needs the whole Compose stack. **Five new dependencies**, pinned and vulnerability-checked. **Branch name** — the skill suggests one, but the project's own DEC-009 wins; that contradiction was later fixed in RFC 0003.

(name)

Roles and responsibilities

Role	Owner
Plan author, implementer	this agent (Claude)
Reviewer, approver, merge decision	ThanhTN

Resources and external actions

Video 01:09. Who owns what, written down before the work starts.

Two corrections, one minute of typing

PROMPT 1

Keep the design plan in there so we can back-track it

RESULT

Revision 1 **rebuilt from the session's own edits** — git could not help, it had never been committed — and restored in full, with a note saying so. But placed **below** revision 2.

PROMPT 2

No — put it in chronological order

RESULT

"Got it: oldest first, so the file reads in the order things happened. I'll **update the saved rule first, then reorder the file.**"

```
- Revision 1: design phase, complete and superseded. It keeps all of its sections, including the Outcome column and the closing note. It still carries the note that I rebuilt it from this session because it was never committed.
- Revision 2: implementation, current. Its content is unchanged. Its opening now says revision 1 is "kept in full above".

Each revision is a top-level section in the file, with its usual sections one level below. Future revisions will be added at the bottom.

I updated the saved rule to match: every revision is kept in full, oldest first, and a new revision goes after the last one rather than above it.

Revision 2 still needs your approval. Once you give it, I'll start with step 1.

* Brewed for 23s · done 11:41 AM
```

✓ Update installed · Restart to update

Video 02:01. "Every revision is kept in full, oldest first, and a new revision goes after the last one."

Approval and a rule change, in one prompt

PROMPT 3

OK, the revision 2 plan seems solid, so approved. Also update the harness

RFC 0002 is written the same way as RFC 0001 — status, affected files, summary, and a motivation section citing this session as evidence.

THEN	WHAT WAS CREATED
Worktree 1	<code>feature/harness-wi002-feedback</code> — RFC 0001 and RFC 0002, one commit each
Worktree 2	<code>feature/WI-002-production-order-screen</code> — the design-phase commit
Main worktree	Cleaned; each copied file verified with <code>git diff --no-index</code> before reverting

```
3 **Status:** adopted
4 **Affected:** 'ai/templates/plan.md', 'ai/skills/planning/SKILL.md', 'ai/evaluations/baseline-cases.md'
5
6
7
8 'plan.md' keeps every revision of a work item's plan, in full and in chronological order. A new revision is appended after the last one. The previous revision is closed in place (Outcome column, Closure Line), and a revision index table at the top marks the current one.
9
10 ## Motivation
11 - +00 Lines
12
13 * Contemplating... (in 1s · + 5.9k tokens)
```

Video 02:26. RFC 0002 — while the editor shows the plan's authorization table.

Diagnosing CI, and preventing a failure

```
• Step 4 is done: 35/35 unit tests pass, including the 4 existing ones. A quick check on the harness PR's CI run:  
  Ran 2 shell commands  
  
• Both CI jobs failed after 1 second. That points to setup, not code: the harness PR only changes Markdown. Checking the logs.  
  Ran 2 shell commands  
  
• CI can't run since GitHub rejected both jobs due to a billing lock on the account — that needs to be cleared on your end. I'll note this as blocked in evidence.md and move on to step 5: adding the two packages needed at the application layer and checking them for known vulnerabilities.  
  Ran 2 shell commands
```

Video 02:52. "That points to setup, not code: the harness PR only changes Markdown."

DIAGNOSED, NOT RETRIED

Both CI jobs failed in one second. Claude reasoned from the content — a Markdown-only PR cannot fail on code — read the logs, and found a **GitHub billing lock**. Recorded as blocked in `evidence.md` rather than treated as a defect.

```
• The solution builds with 0 warnings. Recording the index deviation in DB-002 and decisions.md now, while it's fresh.  
  Ran 2 shell commands  
  
• Moving to step 7: splitting the database logins. First I'll check the current Compose, env file, and git line-ending setup, since CRLF endings in shell scripts can break inside the Postgres container.  
  Ran 2 shell commands  
  
• There's no .gitattributes file, and with core.autocrlf set to true, the new init script would get CRLF line endings and fail on Linux. I'll add a .gitattributes rule to keep *.sh files as LF alongside the script.  
  Ran 2 shell commands  
  
• Now Compose: the database container gets the init script and the new required password setting, and the backend connects as psql_app instead of the owner.
```

Video 03:20. A Windows-to-Linux failure caught before it could happen.

PREVENTED, NOT DEBUGGED

"There's no `.gitattributes` file, and with `core.autocrlf` set to true, the new init script would get CRLF line endings and **fail on Linux**. I'll add a `.gitattributes` rule to keep `*.sh` files as LF alongside the script."

The requirement wrote the test; the test found the bug

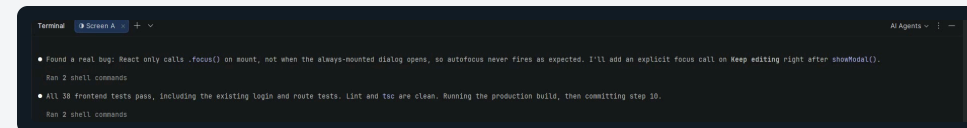
THE COMPONENT CITES ITS REQUIREMENT

// BD-001 items 17-19 (REQ-019, DEC-018). The native <dialog> + showModal() gives the focus trap, Escape and backdrop for free; Escape fires "cancel", which is treated as Keep editing.

A REAL BUG

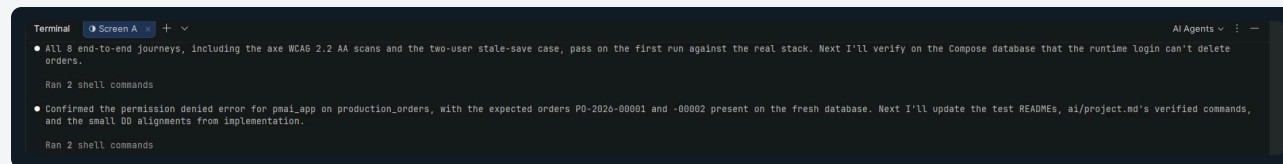
"React only calls `.focus()` on mount, **not when the always-mounted dialog opens**, so `autofocus` never fires as expected. I'll add an explicit focus call on **Keep editing** right after `showModal()`."

REQ-019 said focus must start on *Keep editing*. The component looked correct and was wrong. The only reason it was caught is that a test was written from the requirement.



Video 03:49. Found, explained and fixed — then 38 frontend tests pass.

The least-privilege decision, proved



```
Terminal Screen A + - AI Agents
• All 8 end-to-end journeys, including the axe WCAG 2.2 AA scans and the two-user stale-save case, pass on the first run against the real stack. Next I'll verify on the Compose database that the runtime login can't delete orders.
  Ran 2 shell commands
• Confirmed the permission denied error for pmai_app on production_orders, with the expected orders PO-2026-00001 and -00002 present on the fresh database. Next I'll update the test READMEs, ai/project.md's verified commands, and the small OD alignments from implementation.
  Ran 2 shell commands
```

Video 04:29. DEC-016, closed by attempting the forbidden operation.

"All **8 end-to-end journeys**, including the axe WCAG 2.2 AA scans and the two-user stale-save case, **pass on the first run** against the real stack."

"Confirmed the **permission denied** error for **pmai_app** on **production_orders**, with the expected orders **PO-2026-00001** and **-00002** present on the fresh database."

CAREFUL WITH SOMEONE ELSE'S MACHINE

The database volume was wiped **only because step 7 authorized it**. A fresh **PMAI_APP_DB_PASSWORD** was generated into the worktree's own **.env**, never printed, and checked as gitignored — the user's own **.env** was left untouched.

What an unattended half-hour reports back

KIND	WHAT THE SUMMARY SAID
Deviation	EF Core always adds an index on <code>product_id</code> ; removing it would make migrations drift, so DB-002 was updated instead (DEC-029)
Workaround	Docker Desktop won't mount files from the worktree, so the login script is baked into a small Postgres image — "also more portable"
Defect	The discard dialog wasn't focusing "Keep editing". Fixed and covered by tests
Noise	A Kerberos-library warning from the database driver; harmless, recorded in <code>evidence.md</code>
Limits	E2E isn't in CI yet and runs on Chromium only. No manual screen-reader pass

13	Full verification run from a clean environment; security-review and delivery checklists; <code>evidence.md</code> , <code>status.md</code>	12	security-review, pr-review	evidence, status	Checklists recorded pass/fail with reasons	done 2026-09-18 — final run 03.12Z all green; sr review and delivery checklists pass (<code>evidence.#</code>
14	Push both branches and open two PRs to master (Screen A; harness). Watch the triggered CI runs and record the results. Merge is not done	13 (Screen A); 1 (harness)	pr-review, ci-cd	2 PRs	PR links + CI results in evidence	done 2026-09-18 — PR #3 (Screen A) https://github.com/thantrn95/ProductionManager and PR #2 (harness) opened; CI jobs not started (GitHub billing lock); GitHub secret scan par both; not merged (not authorized)

Done and responsible

Video 05:03. The same plan, hours later: every step now carries its Outcome.

```
Terminal Screen A
- An extra index (DEC-029): EF Core always adds an index on product_id, which DB-002 had left out. Removing it would make the migrations drift, so I kept it and updated DB-002.
- Database init script: Docker Desktop won't mount files from the worktree folder, so the script that creates the restricted login is built into a small Postgres image instead. It's also more portable.
- A real bug the tests caught: the discard dialog wasn't putting focus on "Keep editing" when it opened. It's fixed and covered by tests.
- A harmless log line: the backend container logs a Kerberos-library warning from the database driver. The app works normally; it's recorded in evidence.md.

What you need to do locally:
- Your old database is gone. Your local database volume was wiped, as the plan authorized, and recreated for the Screen A branch.
- Add PMAI_APP_DB_PASSWORD to your deploy/.env. My copy is only in the worktree's .env, and I didn't change yours.
- Migrations now run separately, as the owner login. deploy/README.md has the new order: start the database, run migrations, then start the rest.
- The stack is still running at http://localhost:3000 if you want to try the screen. Sign in as admin with your seed admin password.

Limitations: end-to-end tests aren't in CI yet and run on Chromium only. No manual screen-reader check was done.

Next is your review of #3 and #2. Merging, and removing the two worktrees afterwards, needs your go-ahead.
```

Video 05:03. Built, caught, and still missing — separated.

AND WHAT THE HUMAN MUST DO

"Add `PMAI_APP_DB_PASSWORD` to your `deploy/.env` — my copy is only in the worktree's `.env` , and I didn't change yours." Then: "Merging, and removing the two worktrees afterwards, needs your go-ahead."

The mockup, now running

Home / Production orders / New

New production order

* Required fields are marked with an asterisk.

Order number	Assigned on save		
Status	Draft		
Product *	P-1001 — Steel bracket		
Quantity *	11	Due date *	18/09/2026
Notes	Optional 0/500		

Cancel Save

Video 05:26. `localhost:3000/production-orders/new`, signed in as Seed Admin.

The user squash-merges both pull requests from their own account (**DEC-030**) and opens the running application.

It is the same layout published as a private artifact in the detailed-design step — order number "Assigned on save", status Draft, the 30-product `<select>`, the notes counter, Cancel and Save — now backed by the API, the migration, the seeded database and the restricted `pmai_app` login.

THE GATE HELD

DEC-027 authorized branch, commit, push and open-PR. **Merge was never authorized for the agent**, and it never merged.

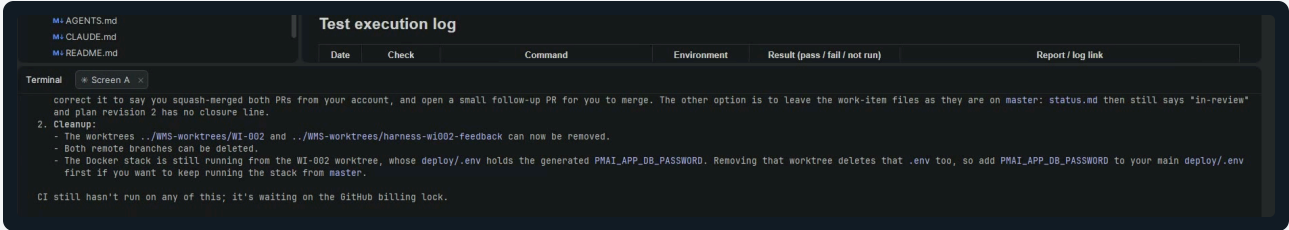
"The merge record is missing, and it's wrong"

After the squash merges, Claude checks what actually landed: "Your local `master` is now at `1eccf9c`, with your `demos/` files untouched. The only difference from PR #3's head is the 12 harness files that came in with #2, so **nothing was lost in the squash.**"

Then it reports a problem **with its own earlier work**: the commit that closes plan revision 2 exists only on the deleted branch, and its text says the merge used merge commits — which is not what happened.

TWO OPTIONS OFFERED, NOT ONE

Correct the record and open a small follow-up PR — or leave it, and accept that `status.md` still says "in-review" and revision 2 has no closure line.



Video 05:47. Cleanup listed too, including which `.env` disappears with the worktree.

RFC 0002 — keep every plan revision, oldest first

The second of the two improvements the Screen A series produced — and the step that also applied the first.

SCOPE 1 — THIS WORK ITEM

Revision 1 rebuilt

It had never been committed, so git could not restore it. Claude reconstructed it from the session's own edits, exactly as approved, and added a note saying so.

SCOPE 2 — THIS AGENT

The rule, then the file

"I'll update the saved rule first, then reorder the file." Every revision kept in full, oldest first, new revisions appended — the same ordering the detailed-design step used.

SCOPE 3 — EVERYONE

RFC 0002, after review

The user asked for the harness itself to be updated.

`ai/templates/plan.md` ,
`ai/skills/planning/SKILL.md` and a new evaluation case.

ROOT CAUSE, AGAIN IN THE WORDING

The plan template said only "If this supersedes a prior revision, state which and why". **Nothing said to keep the old one.** The agent's reading was permitted; the rule was incomplete.

ALSO IN THIS SESSION

RFC 0001 was applied here — four templates, the templates README, the detailed-design skill and a new evaluation case — and committed on its own branch, because harness changes stay out of Screen A's pull request (DEC-028). Those opening minutes are covered by the detailed-design deck, not the cut.

ALSO OBSERVED HERE, FIXED IN RFC 0003

The implementation skill's branch-name example contradicted the project's branch policy, and the Playwright suite ran only locally. Both were closed later.

One screen, four steps, a full paper trail

STEP	AGENT TIME	PRODUCED
01 Basic design	6 m 16 s	Brief, decision log, BD-001
03 Database design	4 m 19 s	DB-002, DEC-013–DEC-020
02 Detailed design	10 m 23 s	4 DD documents + mockup
04 Implementation	~40 m	The screen, 133 tests, 2 PRs

Every requirement from REQ-010 to REQ-019 traces through a BD section, a DB constraint, a DD module, code and at least one test case — and `evidence.md` records each one as **verified**.

Sources: the session recording (`ScreenA_Implementaion.mp4`), its transcript, and the committed files in `work-items/WI-002/` and `ai/improvements/` .

WHAT THE HARNESS GAINED

- **RFC 0001** — always produce all four DD documents.
- **RFC 0002** — keep every plan revision, oldest first.
- Two evaluation cases, so neither regression returns unnoticed.

STILL OPEN AT THE END OF THE RECORDING

CI had not run — a GitHub billing lock. It first executed later, on PR #5, and now runs backend, frontend and E2E on every pull request.